Übungen zu Web Technologies and Applications

Prof. Dr. Jan Stehr

WS 2025/26

Übung 1: Eine kleine Infrastruktur, praktisch mit Vagrant

1. Installieren Sie *VirtualBox* (oder nutzen Sie eine bereits bei Ihnen vorhandene, kompatible Virtualisierungslösung).
2. Installieren Sie *Vagrant*. Bitte beachten: Es gibt immer mal Inkompatibilitäten zwischen Vagrant und Provider der VMs.
3. Legen Sie ein `Vagrantfile` in einem geeigneten Verzeichnis an:

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  config.vm.network "private_network", ip: "10.0.0.10"
end
```

4. Wechseln Sie via Shell in das Verzeichnis, dann `vagrant up`
5. Nehmen Sie ersten Kontakt zur VM über den Internet Stack auf:
   `ping 10.0.0.10`
6. Loggen Sie sich in der VM ein mit `vagrant ssh` und schauen Sie sich die Netzwerkschnittstellen dort an: `ip` mit verschiedenen Parametern, z. B. `address`
7. Was für Prozesse laufen in der VM? `ps` und `ps -ef`
8. Probieren Sie verschiedene Vagrant-Kommandos aus:
   `vagrant`, `vagrant status`, `vagrant box list`…
9. Am Ende `vagrant halt`

Eine kleine Infrastruktur, praktisch mit Vagrant

```
vagrant up
```

```
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'ubuntu/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/bionic64' version '20230607.0.0' is up to date...
==> default: Setting the name of the VM: Lab1_default_1726732279824_59073
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
    default: Adapter 2: hostonly
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:2222
    default: SSH username: vagrant
    default: SSH auth method: private key
    default:
    default: Vagrant insecure key detected. Vagrant will automatically replace
    default: this with a newly generated keypair for better security.
    default:
    default: Inserting generated public key within guest...
    default: Removing insecure key from the guest if it's present...
    default: Key inserted! Disconnecting and reconnecting using new SSH key...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
    default: The guest additions on this VM do not match the installed version of
    default: VirtualBox! In most cases this is fine, but in rare cases it can
    default: prevent things such as shared folders from working properly. If you see
    default: shared folder errors, please make sure the guest additions within the
    default: virtual machine match the version of VirtualBox you have installed on
    default: your host and reload your VM.
    default:
    default: Guest Additions Version: 5.2.42
    default: VirtualBox Version: 7.0
==> default: Configuring and enabling network interfaces...
==> default: Mounting shared folders...
    default: /vagrant => D:/Documents/Lehre/Veranstaltungen/WTA/Lab1
```

Eine kleine Infrastruktur, praktisch mit Vagrant

ping 10.0.0.10

```
Ping wird ausgeführt für 10.0.0.10 mit 32 Bytes Daten:
Antwort von 10.0.0.10: Bytes=32 Zeit<1ms TTL=64
Antwort von 10.0.0.10: Bytes=32 Zeit<1ms TTL=64
Antwort von 10.0.0.10: Bytes=32 Zeit<1ms TTL=64
Antwort von 10.0.0.10: Bytes=32 Zeit<1ms TTL=64

Ping-Statistik für 10.0.0.10:
    Pakete: Gesendet = 4, Empfangen = 4, Verloren = 0
    (0% Verlust),
Ca. Zeitangaben in Millisek.:
    Minimum = 0ms, Maximum = 0ms, Mittelwert = 0ms
```

vagrant ssh

```
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-212-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Thu Sep 19 07:55:03 UTC 2024

  System load:  0.0               Processes:             100
  Usage of /:   3.0% of 38.70GB   Users logged in:       0
  Memory usage: 13%               IP address for enp0s3: 10.0.2.15
  Swap usage:   0%                IP address for enp0s8: 10.0.0.10


Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


vagrant@ubuntu-bionic:~$
```

Eine kleine Infrastruktur, praktisch mit Vagrant

In der VM:

```
vagrant@ubuntu-bionic:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::3b:7bff:feb7:3b2d  prefixlen 64  scopeid 0x20<link>
        ether 02:3b:7b:b7:3b:2d  txqueuelen 1000  (Ethernet)
        RX packets 1311  bytes 442871 (442.8 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 850  bytes 147135 (147.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.10  netmask 255.255.255.0  broadcast 10.0.0.255
        inet6 fe80::a00:27ff:fee0:5e74  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:e0:5e:74  txqueuelen 1000  (Ethernet)
        RX packets 21  bytes 1796 (1.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 18  bytes 1352 (1.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 16  bytes 1576 (1.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 16  bytes 1576 (1.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
vagrant@ubuntu-bionic:~$ ps
  PID TTY          TIME CMD
 2719 pts/0    00:00:00 bash
 2772 pts/0    00:00:00 ps
```

```
vagrant@ubuntu-bionic:~$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 07:51 ?        00:00:00 /sbin/init
root         2     0  0 07:51 ?        00:00:00 [kthreadd]
root         4     2  0 07:51 ?        00:00:00 [kworker/0:0H]
...
root      1390     1  0 07:51 ?        00:00:00 /usr/sbin/cron -f
root      1402     1  0 07:51 ?        00:00:00 /usr/lib/accountsservice/accounts-
daemon
root      1484     1  0 07:51 ttyS0    00:00:00 /sbin/agetty -o -p -- \u --keep-
baud 115200,38400,9600 ttyS0 vt220
root      1578     1  0 07:51 tty1     00:00:00 /sbin/agetty -o -p -- \u --noclear
tty1 linux
root      1593     1  0 07:51 ?        00:00:00 /usr/lib/policykit-1/polkitd --no-
debug
root      1603     1  0 07:51 ?        00:00:00 /usr/sbin/VBoxService
root      1717     1  0 07:51 ?        00:00:00 /usr/sbin/sshd -D
systemd+  2465     1  0 07:51 ?        00:00:00 /lib/systemd/systemd-networkd
root      2629     2  0 07:52 ?        00:00:00 [kworker/1:4]
root      2641  1717  0 07:55 ?        00:00:00 sshd: vagrant [priv]
vagrant   2643     1  0 07:55 ?        00:00:00 /lib/systemd/systemd --user
vagrant   2644  2643  0 07:55 ?        00:00:00 (sd-pam)
vagrant   2718  2641  0 07:55 ?        00:00:00 sshd: vagrant@pts/0
vagrant   2719  2718  0 07:55 pts/0    00:00:00 -bash
root      2759     2  0 07:58 ?        00:00:00 [kworker/u4:1]
root      2760     2  0 07:58 ?        00:00:00 [kworker/1:0]
vagrant   2771  2719  0 07:59 pts/0    00:00:00 ps -ef
```

Eine kleine Infrastruktur, praktisch mit Vagrant

vagrant status

```
Current machine states:

default                   running (virtualbox)

The VM is running. To stop this VM, you can run `vagrant halt` to
shut it down forcefully, or you can run `vagrant suspend` to simply
suspend the virtual machine. In either case, to restart it again,
simply run `vagrant up`.
```

vagrant box list

```
bento/ubuntu-18.04            (virtualbox, 202107.07.0)
bento/ubuntu-18.04            (virtualbox, 202107.28.0)
generic/ubuntu2110            (virtualbox, 4.2.16)
hashicorp/bionic64            (virtualbox, 1.0.282)
rapid7/metasploitable3-ub1404 (virtualbox, 0.1.12-weekly)
rapid7/metasploitable3-win2k8 (virtualbox, 0.1.0-weekly)
ubuntu/bionic64               (virtualbox, 20230607.0.0)
```

Übung 1: Eine kleine Infrastruktur, praktisch, alternativ mit Docker

Sollte Vagrant nicht funktionsfähig/gewünscht/nötig sein:

1. Installieren Sie Docker (oder einen kompatiblen Verwandten davon).
2. Legen Sie eine Datei `docker-compose.yml` in einem geeigneten Verzeichnis an.
3. Starten Sie den Container mit `docker compose up -d` (Parameter d für *detached*).
4. Schauen Sie sich das Netzwerk zwischen den Containern an: `docker network ls` und `docker network inspect`
5. Starten Sie eine Shell in einem Container mit `docker exec -it container1 /bin/bash`
6. Was für Prozesse laufen im Container? Und außerhalb? `ps` und `ps -ef`
7. Wie sieht es mit Namespaces aus? `lsns`
8. Schauen Sie sich die Netzwerkschnittstellen mit `ifconfig` an. `ping`-en Sie den anderen Container über das Netz zwischen beiden an.
9. Verlassen mit `exit`, Container beenden mit `docker compose down`

docker-compose.yml

```yaml
services:
  container1:
    image: ubuntu:latest
    container_name: container1
    networks:
      small_internal_network:
        ipv4_address: 10.0.0.10
    command: /bin/bash -c "apt-get update && apt-get install -y
iputils-ping net-tools && /bin/bash;while true; do sleep 30;
done"
    tty: true

  container2:
    image: ubuntu:latest
    container_name: container2
    networks:
      small_internal_network:
        ipv4_address: 10.0.0.11
    command: /bin/bash -c "while true; do sleep 30; done"
    tty: true

networks:
  small_internal_network:
    driver: bridge
    ipam:
      config:
        - subnet: 10.0.0.0/24
```

Eine kleine Infrastruktur, praktisch, alternativ mit Docker

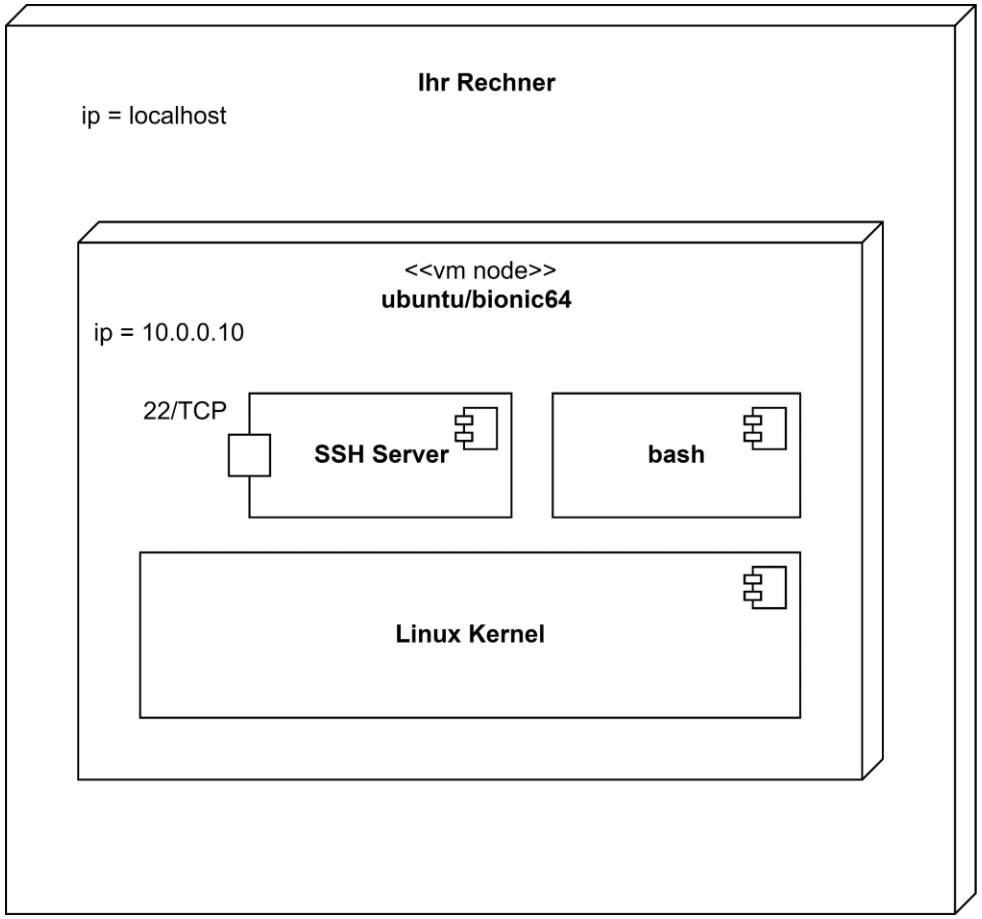
```
root@jan-VirtualBox:/home/jan/Dokumente/Veranstaltungen/wta/lab1# docker compose up -d
[+] Running 2/3
 ⠿ Network lab1_small_internal_network  Created                                                    0.3s
 ✔ Container container2                 Started                                                    0.3s
 ✔ Container container1                 Started                                                    0.2s
root@jan-VirtualBox:/home/jan/Dokumente/Veranstaltungen/wta/lab1# docker network ls
NETWORK ID     NAME                          DRIVER    SCOPE
db97ec37e12c   bridge                        bridge    local
97335d471bc2   host                          host      local
195cc3b44223   lab1_small_internal_network   bridge    local
a133db8b4535   none                          null      local
root@jan-VirtualBox:/home/jan/Dokumente/Veranstaltungen/wta/lab1# docker network inspect lab1_small_internal_network
[
    {
        "Name": "lab1_small_internal_network",
        "Id": "195cc3b4422368f7a87ccd266c764f49c46c538fd309fc87e22de37806ef3d69",
        "Created": "2024-09-19T13:13:53.293091276+02:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "10.0.0.0/24"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "1b755ee133ee7d1e297b7de42a2480a0e034a196514cada78fc7145953875a30": {
                "Name": "container1",
                "EndpointID": "135ffa82d81c08f68b443da21ac2caf79bcb0800470dcdd592cc5e86a41b72f5",
                "MacAddress": "02:42:0a:00:00:0a",
                "IPv4Address": "10.0.0.10/24",
                "IPv6Address": ""
            },
            "c3ddb0dbc5ddf4eab893876ed2a4cd9a7c95396097a17ecde3ee153ef24eb7fa": {
                "Name": "container2",
```

Eine kleine Infrastruktur, praktisch, alternativ mit Docker

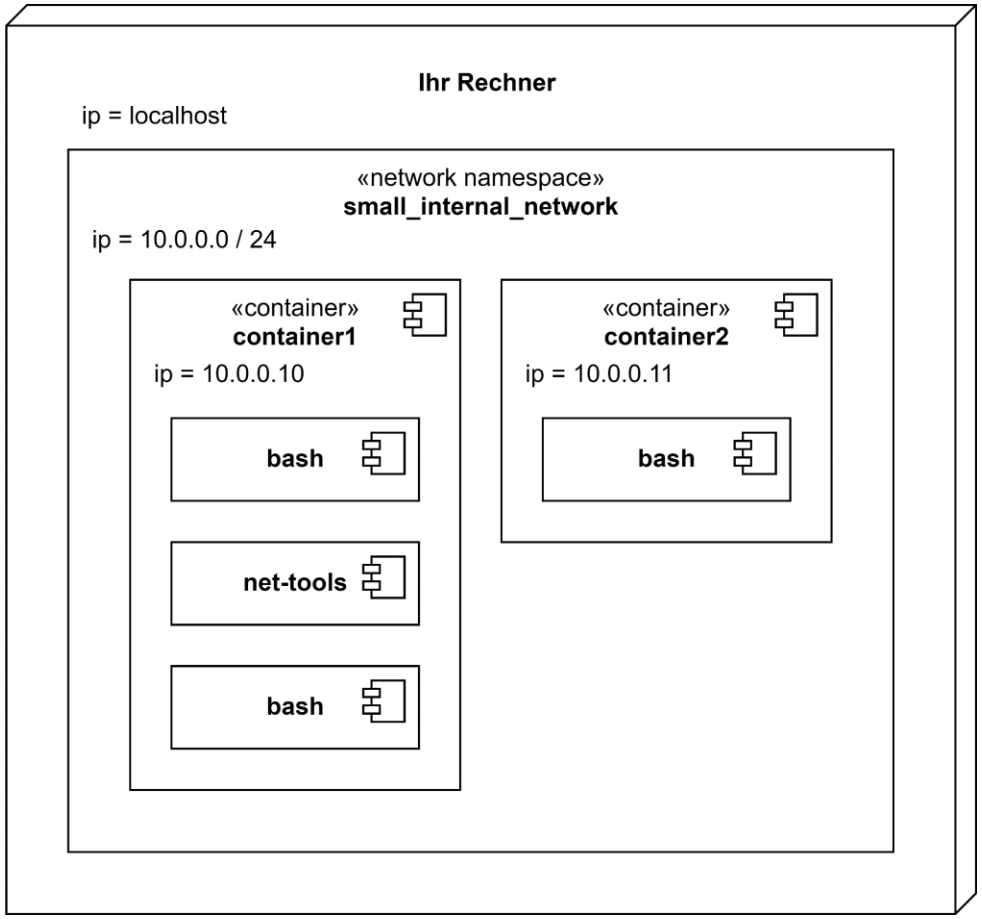
```
root@jan-VirtualBox:/home/jan/Dokumente/Veranstaltungen/wta/lab1# docker exec -it container1 /bin/bash
root@1b755ee133ee:/# ls
bin                 boot  etc   lib    media  opt   root  sbin                srv  tmp  var
bin.usr-is-merged   dev   home  lib64  mnt    proc  run   sbin.usr-is-merged  sys  usr
root@1b755ee133ee:/# ps
    PID TTY          TIME CMD
    208 pts/1    00:00:00 bash
    217 pts/1    00:00:00 ps
root@1b755ee133ee:/# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 11:13 pts/0    00:00:00 /bin/bash -c apt-get update && apt-get install -y iputils-ping net
root         205       1  0 11:15 pts/0    00:00:00 /bin/bash
root         208       0  0 11:17 pts/1    00:00:00 /bin/bash
root         218     208  0 11:18 pts/1    00:00:00 ps -ef
root@1b755ee133ee:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.0.10  netmask 255.255.255.0  broadcast 10.0.0.255
        ether 02:42:0a:00:00:0a  txqueuelen 0  (Ethernet)
        RX packets 3680  bytes 25642774 (25.6 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3551  bytes 197269 (197.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 986 (986.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 986 (986.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@1b755ee133ee:/# ping 10.0.0.11
PING 10.0.0.11 (10.0.0.11) 56(84) bytes of data.
64 bytes from 10.0.0.11: icmp_seq=1 ttl=64 time=0.104 ms
64 bytes from 10.0.0.11: icmp_seq=2 ttl=64 time=0.115 ms
64 bytes from 10.0.0.11: icmp_seq=3 ttl=64 time=0.118 ms
64 bytes from 10.0.0.11: icmp_seq=4 ttl=64 time=0.108 ms
64 bytes from 10.0.0.11: icmp_seq=5 ttl=64 time=0.117 ms
64 bytes from 10.0.0.11: icmp_seq=6 ttl=64 time=0.115 ms
64 bytes from 10.0.0.11: icmp_seq=7 ttl=64 time=0.116 ms
^C
--- 10.0.0.11 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6200ms
rtt min/avg/max/mdev = 0.104/0.113/0.118/0.004 ms
root@1b755ee133ee:/#
```

Unterschiede zwischen den beiden Infrastrukturen

VM mit Vagrant



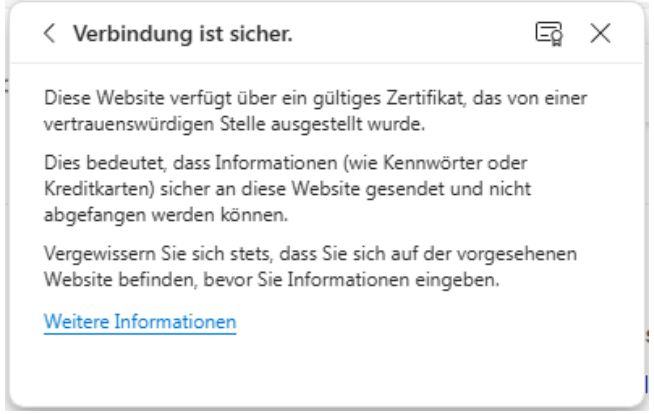
Container und Netzwerk mit Docker



Übung 2: Verständnisfragen zu TLS

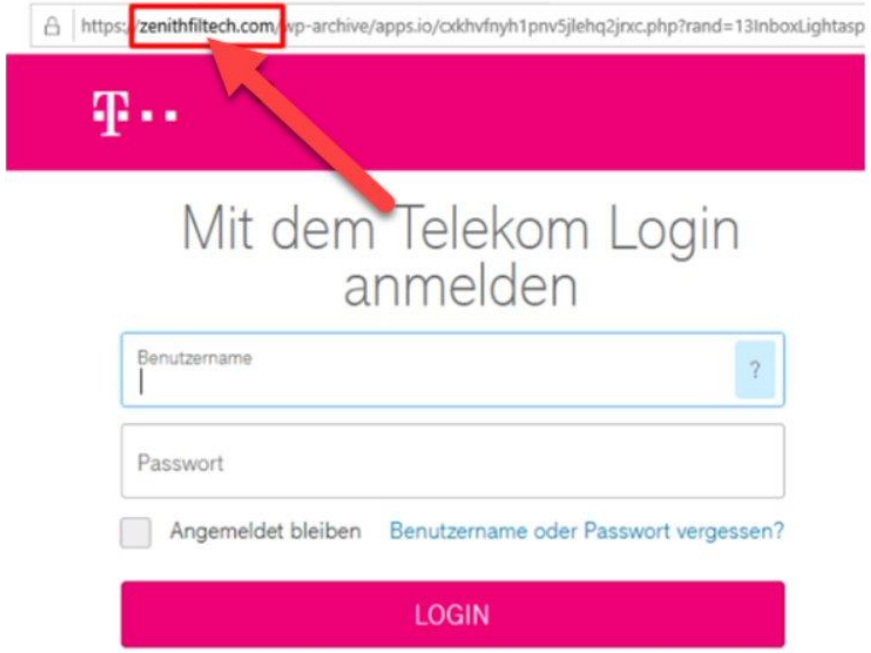
Welche Aussagen sind richtig oder falsch und warum?

1. Die ausgetauschten Daten zwischen meinem Browser und dem Server sind durch TLS für Dritte nicht lesbar. Es sieht z. B. also niemand, was ich auf einer Website mache.
2. Das Zertifikat des Servers überprüft mein Client bei einer Zertifizierungsstelle (CA; Certificate Authority). Dadurch weiß ich genau, dass ich mit dem richtigen Server kommuniziere, z. B. dem meiner Bank.



TLS: Verständnisfragen

1. Bei der Interaktion mit Websites kommt es trotz TLS auf die Zugriffsarten an. Auf bestimmte Operationen kann aus den Metadaten geschlossen werden (siehe Kapitel HTTP). Außerdem ist die Frage, wo die TLS Termination geschieht und was danach erfolgt und wo.
2. Auch eine „offizielle“, „anerkannte“ oder „etablierte“ CA muss nicht grundsätzlich vertrauenswürdig sein. Es kommt z. B. drauf an, wer wir sind und mit wem die CA assoziiert ist. Außerdem: Sobald das Zertifikat empfangen wurde, kann es z. B. jemand anderes für eine Phishing Website nutzen.



Bildquelle: https://www.mc50plus.de/2020/05/26/gefaelschte-webseiten-erkennen/

Übung 3: Praktische Nutzung von DNS

Eine DNS-Abfrage können wir auch manuell mit `nslookup` oder `dig` auf der Kommandozeile durchführen, z. B.:

`nslookup fhdw.de` – „nicht autorisierende Antwort“ folgt darauf (die ersten beiden Zeilen sind Ihr DNS Resolver).

`nslookup –type=NS fhdw.de` – verantwortliche Nameserver finden.

1. Fragen Sie Informationen über eine Website Ihrer Wahl mit den unterschiedlichen Parametern ab (z. B. den RR Types vorne für `-type`, `-debug`, oder weiteren über `-?` oder `--help`).
2. Was bedeutet es, wenn wir mehrere IP-Adressen für eine Domain bekommen? Wozu dient DNS auch?
3. Was sind potentielle Probleme, z. B. mit Sicherheit und Verfolgbarkeit, bei DNS in der einfachen, vorgestellten Form?

Beispiel DNS Root Zone

```
nslookup -type=NS .

Nicht autorisierende Antwort:
(root)  nameserver = h.root-servers.net
(root)  nameserver = e.root-servers.net
(root)  nameserver = b.root-servers.net
(root)  nameserver = c.root-servers.net
(root)  nameserver = d.root-servers.net
(root)  nameserver = i.root-servers.net
(root)  nameserver = f.root-servers.net
(root)  nameserver = k.root-servers.net
(root)  nameserver = j.root-servers.net
(root)  nameserver = m.root-servers.net
(root)  nameserver = a.root-servers.net
(root)  nameserver = g.root-servers.net
(root)  nameserver = l.root-servers.net

a.root-servers.net      AAAA IPv6 address = 2001:503:ba3e::2:30
b.root-servers.net      AAAA IPv6 address = 2801:1b8:10::b
c.root-servers.net      AAAA IPv6 address = 2001:500:2::c
d.root-servers.net      AAAA IPv6 address = 2001:500:2d::d
e.root-servers.net      AAAA IPv6 address = 2001:500:a8::e
f.root-servers.net      AAAA IPv6 address = 2001:500:2f::f
g.root-servers.net      AAAA IPv6 address = 2001:500:12::d0d
h.root-servers.net      AAAA IPv6 address = 2001:500:1::53
i.root-servers.net      AAAA IPv6 address = 2001:7fe::53
```

```
nslookup -type=SOA .

Nicht autorisierende Antwort:
(root)
        primary name server = a.root-servers.net
        responsible mail addr = nstld.verisign-grs.com
        serial  = 2025011700
        refresh = 1800 (30 mins)
        retry   = 900 (15 mins)
        expire  = 604800 (7 days)
        default TTL = 86400 (1 day)
```

DNS: Eine Domain mit mehreren IP-Adressen

- **Load Balancing** – Domain nutzt mehrere Server mit den gleichen Diensten, um den Traffic zu verteilen und ressourcenoptimiert zu verarbeiten.
- **Redundanz** – mehrere Server hinter einer Domain, um die Dienste dahinter immer (hoch-)verfügbar zu halten.
- **Content Delivery Networks (CDN)** – geographisch verteilte Datacenter, Domäne wird möglichst nahe an Kunden bedient, IP-Adresse mit der performantesten Verbindung nutzen.
- **Service Discovery** – verschiedene Dienste und Anwendungen einer Domäne auf verschiedenen Hosts, z. B. MS365.

DNS und Sicherheit

DNS übersetzt mit im Internet verteilten Diensten unsere Host-Aufrufe in konkrete IP-Adressen. Spontan finden wir zwei Sicherheitsprobleme:

1. **DNS Spoofing**, **Cache Poisoning** – leitet uns auf Web-Seiten mit Malware um. Betrifft also *Authentizität* und *Integrität* des Dienstes.
2. **Verfolgung von Web-Aufrufen** – da TLS zwar unseren ausgetauschten Content verschlüsselt, aber die Betreiber der DNS Server sehen, welche Domänen und Hosts wir aufrufen. Betrifft also die *Vertraulichkeit* des Dienstes.

Für Punkt 1 gibt es DNSSEC (DNS Security Extensions), was aber nur einen Teil der Probleme angeht. Und Vertraulichkeit kommt dort nicht vor. Das kann durch DNS über TLS/HTTPS zumindest auf dem Transport unterstützt werden.

Übung 4: Wir bauen uns einen kleinen Webserver

Unsere minimale virtuelle Infrastruktur bauen wir jetzt aus: Wir lassen (die VM) einen Container mit einem Webserver ausführen. Also, wir „deployen“ einen Container auf unserem (virtuellen) Knoten. Legen Sie in einem passenden Verzeichnis die Dateien unten an. Wenn der Webserver läuft, machen Sie ihn ansprechbar z. B. unter `wta.meinedomain.com` (Hinweis in Abschnitt DNS vorne).

webserver.py

```python
from flask import Flask

webserver = Flask(__name__)

@webserver.route('/')
def home():
  return '<h1>Willkommen auf der minimalen WTA-Homepage!</h1>'

if __name__ == '__main__‘:
  webserver.run(host='0.0.0.0', port=8000)
```

In einem Python-Programm setzen wir die Bibliothek *Flask* ein, die uns einen Webserver implementiert.

Über *Decorators* @ können wir sehr einfach Routen auf unserer Website auf Funktionen abbilden, die bei HTTP Request des Pfades ausgeführt werden.

Nutzen Sie gerne auch andere Implementierungen in Go oder Java-/Typescript.

Dockerfile

```dockerfile
FROM python:3.9.20-slim

RUN pip install flask
RUN mkdir /server
COPY webserver.py /server
WORKDIR /server

CMD ["python", "webserver.py"]
```

Wenn direkt als Container, ohne Vagrant/VM (ohne Zeilenumbrüche!):

`docker buildx build –t webserver .`

`docker run –p 8001:8000 webserver`

`curl -v localhost:8001`

Vagrantfile

```vagrant
Vagrant.configure("2") do |config|

  config.vm.box = "ubuntu/jammy64"
  config.vm.provision :docker

  config.vm.network "private_network", ip: "10.0.0.10"

  config.vm.provision "docker" do |d|
    d.build_image "/vagrant", args: "-t webserver"
    d.run "webserver", args: "-p 8001:8000"
  end

end
```

Um komplexere, verteilte Infrastrukturen lokal zu entwickeln und zu testen, bieten sich VMs an, denen wir lokale Adressen zuweisen und in denen wir die Applikationscontainer dann ausführen.

`vagrant up`

`curl -v 10.0.0.10:8001` – was sehen Sie?

Loggen Sie sich wieder in die VM ein und schauen Sie, was dort läuft. Auch mit **`docker ps`**

Wie sieht das entstandene Deployment als Diagramm aus?

Wir bauen uns einen kleinen Webserver

```
curl -v 10.0.0.10:8001
*   Trying 10.0.0.10:8001...
* Connected to 10.0.0.10 (10.0.0.10) port 8001
> GET / HTTP/1.1
> Host: 10.0.0.10:8001
> User-Agent: curl/8.9.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: Werkzeug/3.0.4 Python/3.9.20
< Date: Wed, 13 Nov 2024 11:16:24 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 51
< Connection: close
<
<h1>Willkommen auf der minimalen WTA-Homepage!</h1>* shutting down connection #0
```

Ergänze ich in der Datei `/etc/hosts` die Zeile

```
10.0.0.10    wta.meinedomain.stehr
```

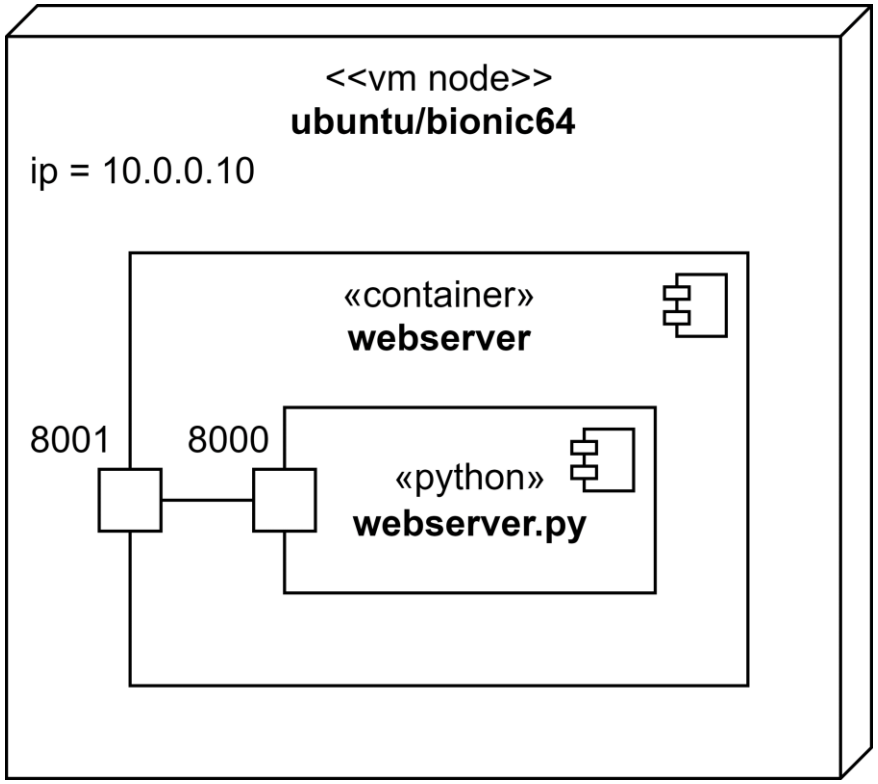
nimmt das DNS nun lokal die Namensauflösung vor:

```
curl wta.meinedomain.stehr:8001
<h1>Willkommen auf der minimalen WTA-Homepage!</h1>
```

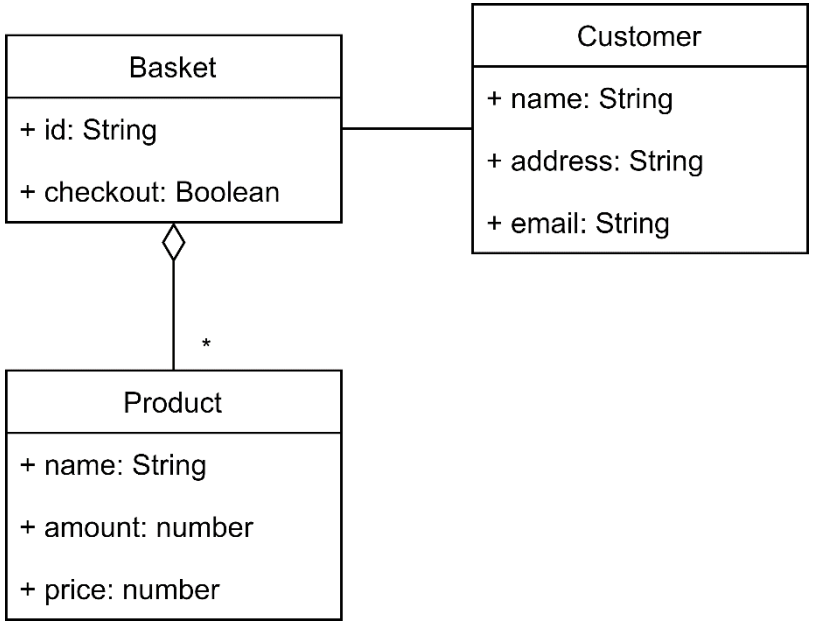
Das ist hilfreich für die Identifikation lokaler Ressourcen und Dienste in virtuellen Infrastrukturen (z. B. für Service Discovery).
Warum übernimmt der Browser das i. d. R. nicht so einfach?

```
vagrant ssh
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-213-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Wed Nov 13 11:16:56 UTC 2024

  System load:  0.41              Users logged in:        0
  Usage of /:   5.4% of 38.70GB   IP address for enp0s3:  10.0.2.15
  Memory usage: 20%               IP address for enp0s8:  10.0.0.10
  Swap usage:   0%                IP address for docker0: 172.17.0.1
  Processes:    106

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Infrastructure is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

119 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 18.04 at
https://ubuntu.com/18-04

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Sat Sep 14 14:09:27 2024 from 10.0.2.2
vagrant@ubuntu-bionic:~$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED       STATUS
PORTS                                       NAMES
33074aed9a46   webserver   "python webserver.py"   8 weeks ago   Up About a minute
0.0.0.0:8001->8000/tcp, :::8001->8000/tcp   webserver
```

Ein kleiner Webserver: Deployment-Diagramm



Übung 5: JSON



Legen Sie zu der Warenkorb-Klassen-Struktur links eine beispielhafte JSON-Struktur an (einfach als Text, oder in einer Programmiersprache Ihrer Wahl).

Alternative 1: Ein einzelner Warenkorb.

Alternative 2: Eine Struktur mit mehreren Warenkörben, auf die man über ihre ID zugreifen kann.

Warenkorb in JSON

Alternative 1: Ein einzelner Warenkorb.

```
{
    "id": "basket001",
    "checkout": true,
    "Customer": {
        "name": "Test Tester",
        "address": "Teststrasse 1, 12345 Teststadt",
        "email": "test@test.com"
    },
    "products": [
        {
            "name": "product1",
            "amount": 10,
            "price": 50.0
        },
        {
            "name": "product2",
            "amount": 1,
            "price": 10.0
        }
    ]
}
```

Alternative 2: Eine Struktur mit mehreren Warenkörben, auf die man über ihre ID zugreifen kann.

```
{
    "basket001": {
        "checkout": true,
        "Customer": {
            "name": "Test Tester",
            "address": "Teststrasse 1, 12345 Teststadt",
            "email": "test@test.com"
        },
        "products": [
            {
                "name": "product1",
                "amount": 10,
                "price": 50.0
            },
            {
                "name": "product2",
                "amount": 1,
                "price": 10.0
            }
        ]
    },
    "basket002": {
        "checkout": false,
        "Customer": {
            "name": "Another Tester",
            "address": "Testweg 1, 54321 Testdorf",
            "email": "another@test.com"
        },
        "products": [
            {
                "name": "product3",
                "amount": 20,
                "price": 5.5
            },
            {
                "name": "product2",
                "amount": 2,
                "price": 20.0
            }
        ]
    }
}
```

Übung 6: Ein Webserver schickt HTML-Seiten

Unseren Webserver gestalten wir nun etwas um: Er soll uns statische HTML-Seiten aus Verzeichnissen schicken.

webserver.py

```python
from flask import Flask, send_from_directory

webserver = Flask(__name__)

@webserver.route('/')
def home():
    return send_from_directory('root-page',
'index.html')

@webserver.route('/<path:path>')
def static_pages(path):
    return send_from_directory('root-page',
path)

if __name__ == '__main__':
    webserver.run(host='0.0.0.0', port=8000)
```

Dockerfile

```dockerfile
FROM python:3.9.20-slim

RUN pip install flask
RUN mkdir /server
COPY ./webserver.py /server
COPY ./root-page /server/root-page
WORKDIR /server

CMD ["python", "webserver.py"]
```

Vagrantfile

```vagrant
Vagrant.configure("2") do |config|

  config.vm.box = "ubuntu/jammy64 "
  config.vm.provision :docker

  config.vm.network "private_network", ip: "10.0.0.10"
  config.vm.provision "shell", inline: <<-SHELL
    mkdir ./root-page
    cp -r /vagrant/root-page/* ./root-page/
  SHELL

  config.vm.provision "docker" do |d|
    d.build_image "/vagrant", args: "-t webserver"
    d.run "webserver", args: "-p 8001:8000"
  end

end
```

/root-page/index.html

```html
<!DOCTYPE html>
<html lang="de">
<head>
    <title>Ein Webserver im Container</title>
</head>
<body>
    <h1>Hallo sagt der Python Flask
Webserver!</h1>
    <p>Das ist eine einfache, statische HTML-
Seite, geschickt von Flask in einem
Container.</p>
</body>
</html>
```

/root-page/pages/page1.html

```html
<!DOCTYPE html>
<html lang="de">
<head>
    <title>Weitere Seite</title>
</head>
<body>
    <h1>Eine weitere Beispielseite in
HTML</h1>
    <p>Das ist noch eine einfache,
statische HTML-Seite, geschickt von
Flask in einem Container.</p>
</body>
</html>
```

In Ihrem Arbeitsverzeichnis von Vagrant oder Docker brauchen Sie hier nun ein Unterverzeichnis `/root-page` mit der Struktur von HTML-Seiten für den Webserver.

Ein Webserver schickt HTML-Seiten

Siehe Quellcodes zu Übung 6 auf Teams.

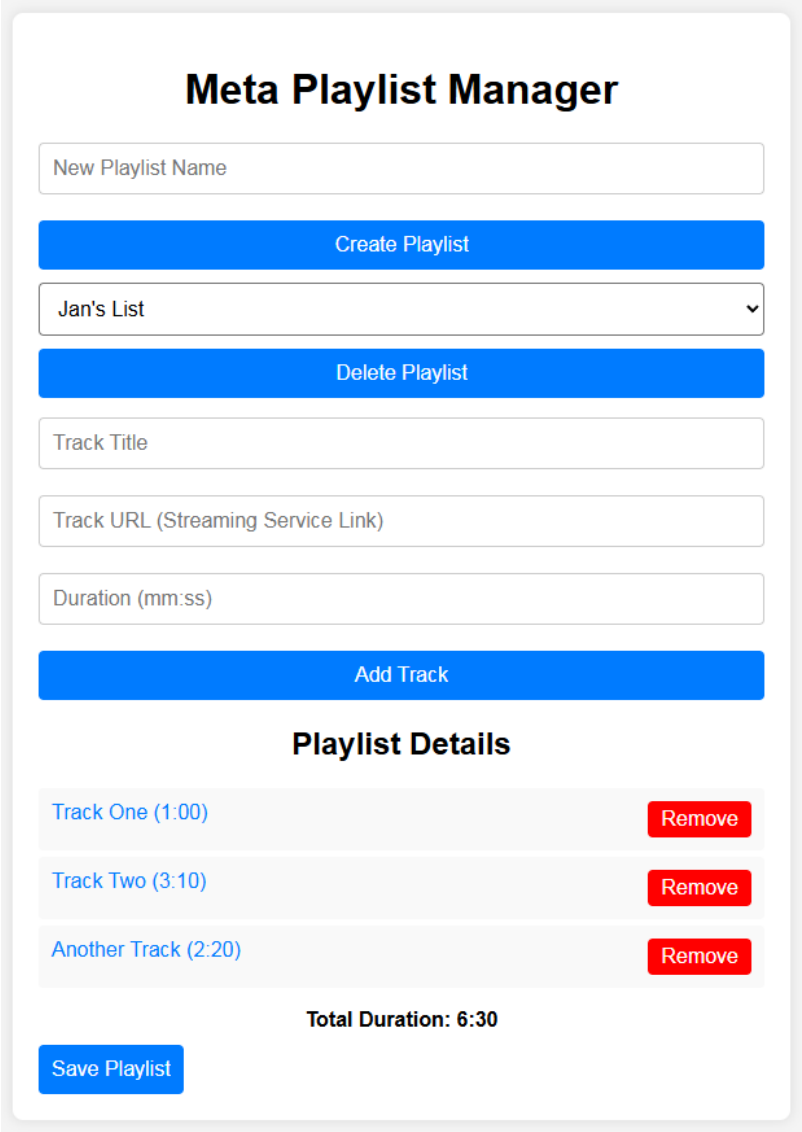
Übung 7: Eine Oberfläche für eine responsive Web App

Implementieren Sie ein **User Interface** für eine **Meta Playlist App**, mit der wir Musik von mehreren Streaming-Diensten verwalten können.

1. Implementieren Sie die notwendigen **Elemente** mit **HTML**.
2. Implementieren Sie ein passendes **Layout** mit **CSS**. Es soll responsiv sein, sich also auf Desktop und Mobilgeräten anpassen.

Sie können das UI selbst schreiben, auf eine Gruppe aufteilen, und/oder einen AI-Assistenten nutzen.

Im Job und in der Klausur müssen Sie aber wissen, wie das Produkt funktioniert, es erklären und Modifikationen beschreiben können.



Eine Oberfläche für eine responsive Web App

Siehe Quellcodes zu Übung 7 auf Teams.

Übung 8: Das Verhalten unserer responsiven Web App

Erweitern Sie die Meta Playlist App mit JavaScript:

1. Fügen Sie *EventListener* für Funktionen hinzu, die auf die einzelnen Buttons reagieren.
2. Implementieren Sie diese *Funktionen* zum *Anlegen* und *Löschen* einer Playlist, *Hinzufügen* und *Löschen von Songs*, *Berechnung der Gesamtspielzeit*.
3. Verwalten Sie die Playlists intern *in Form von JSON*.
4. *Speichern* Sie die JSON-Struktur *lokal*.

Das Verhalten unserer responsiven Web App

Siehe Quellcodes zu Übung 8 auf Teams.

Übung 9: Ein Server für unsere Meta Playlist App

Wir wollen unsere Playlists nicht mehr lokal, sondern auf einem Server speichern. Außerdem wollen wir die Playlist-Verwaltung als Dienst anbieten, der von unserer Web App, aber auch von anderen Clients benutzt werden kann.

1. Spezifizieren Sie für den Server eine REST API im Format von OpenAPI mit folgenden Möglichkeiten: Alle Playlists abrufen, eine Playlist anlegen oder aktualisieren, eine bestimmte Playlist abrufen, eine Playlist löschen. Die Daten sollen in JSON ausgetauscht werden.
2. Ergänzen Sie den Webserver um die passenden Routen und Funktionen. Die Playlists speichern Sie zunächst einfach lokal in einer Variable im Speicher des Servers als JSON-Struktur, wie wir das vorher schon im Client gemacht haben. Den Client liefert der Server als HTML-Dokument zurück, z. B. direkt als `index.html` auf der Default Route `/`.
3. Stellen Sie den Webserver als Container bereit und testen Sie ihn auf geeignete Weise (*curl, Swagger, Postman, o. ä.*) zunächst gesondert als Subsystem.
4. Passen Sie die Client App auf die Schnittstelle des Servers an und testen Sie die Integration.

Sie müssen nicht unbedingt alles ausimplementieren. Wichtig ist es, dass Sie die Konzepte, Funktionsweisen und Abläufe praktisch nachvollziehen.

Ein Server für unsere Meta Playlist App

Siehe Quellcodes zu Übung 9 auf Teams.

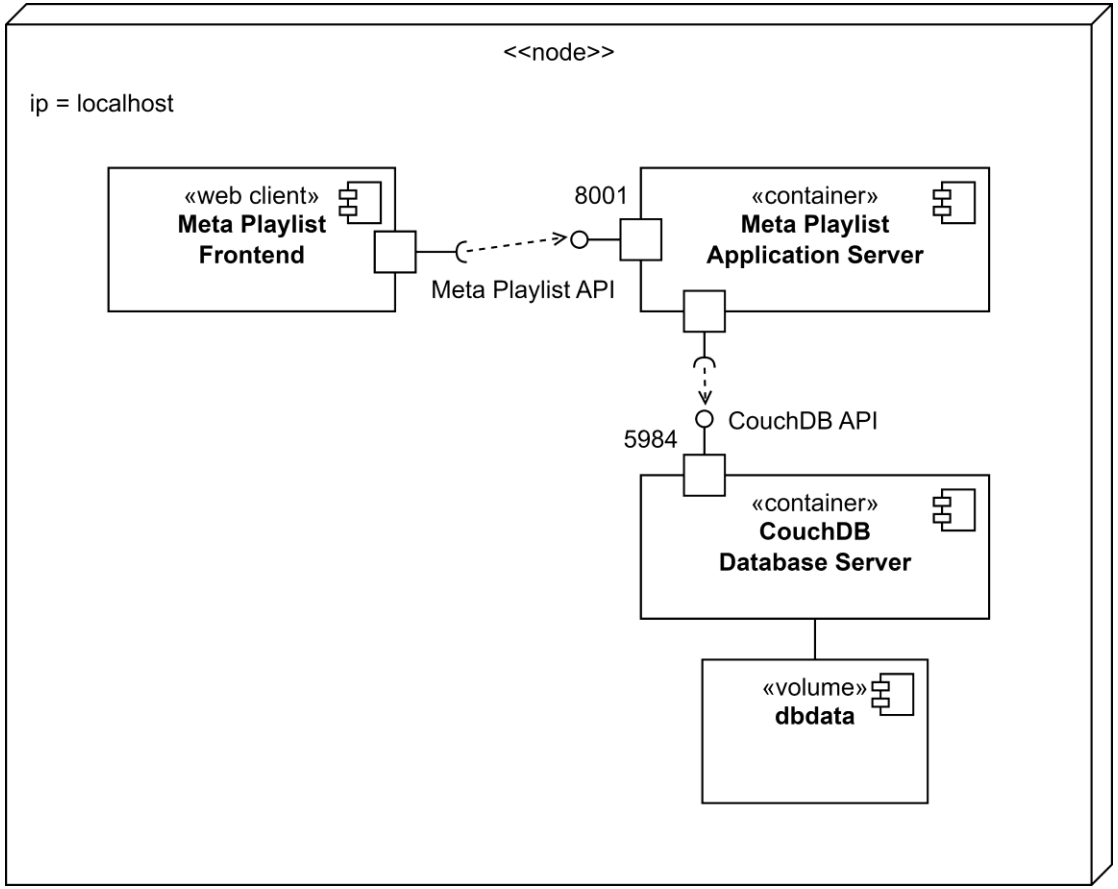
Übung 10: Eine Datenhaltung für unseren Web-Dienst

Wir wollen unsere Playlists in einer Datenhaltung ablegen. Dazu binden wir die dokumentenorientierte Datenbank *CouchDB* (`couchdb.apache.org`) an unseren Webserver an.

1. CouchDB nutzen wir als Container, die Datenbank wird über eine REST API angesprochen. Es gibt ein fertiges Image, das wir einfach zusammen mit unserem Webserver in ein `docker-compose.yml` integrieren. Diese Datei, mit einigen abhängigen Dateien, ist als Ausgangspunkt gegeben (liegt auf Teams). Wir gehen die zunächst einmal gemeinsam durch.
2. Der Webserver muss nun die JSON-Objekte der Playlists über die REST API an die CouchDB übergeben bzw. von dort abfragen. Die Funktionen des Webservers modifizieren wir entsprechend – finden Sie heraus, wie das geht. Den Client sollte der Webserver wie in Übung 9 liefern.
3. Testen Sie die Gesamtlösung auf geeignete Weisen, also z. B. den Server über seine REST API, dann zusammen mit dem Client. Beantworten Sie dabei folgende Fragen:
   a. Wie genau findet sich hier das Microservice-Konzept umgesetzt?
   b. Welches wichtige Element läuft im CouchDB-Container neben der Datenbank selbst?
   c. Wie sieht die Architektur aus, welche Komponenten gibt es, über welche Schnittstellen kommunizieren die?

Eine Datenhaltung für unseren Web-Dienst

Siehe Quellcodes zu Übung 10 auf Teams.



Übung 11: K8s zum Ausprobieren

1. Installieren Sie **Minikube**: `https://minikube.sigs.k8s.io/`
2. Installieren Sie **kubectl**: `https://kubernetes.io/docs/tasks/tools/`
3. Starten Sie Ihren K8s Cluster mit `minikube start`
   Unter MacOS und Linux sollten Sie mit `--driver=docker` die Control Plane im Container nutzen.
4. Orientieren Sie sich auf dem Dashboard mit `minikube dashboard` und finden Sie die Informationen zu Ihrem lokalen Node.
   Alternativ auf der Kommandozeile:
   ```
   kubectl get nodes
   kubectl describe node [NAME]
   ```
5. Warum bezeichnet man K8s als „Betriebssystem der Cloud“? Welches sind die Analogien?

Falls Sie alternative Installationen zum lokalen Testen ausprobieren möchten, gibt es z. B. `https://devopscube.com/kubernetes-cluster-vagrant/`

K8s zum Ausprobieren

jan.stehr — -zsh — 147×50

```
(base) jan.stehr@mac-K575TX3J5F ~ % minikube start --driver=docker
😄  minikube v1.34.0 auf Darwin 15.2 (arm64)
✨  Verwende den Treiber docker basierend auf dem existierenden Profil
👍  Starte "minikube" primary control-plane Node im "minikube" Cluster
🚜  Ziehe Base Image v0.0.45 ...
🔄  Starte existierenden docker container für "minikube" ...
🐳  Vorbereiten von Kubernetes v1.31.0 auf Docker 27.2.0...
🔎  Verifiziere Kubernetes Komponenten...
    ▪ Verwende Image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Verwende Image gcr.io/k8s-minikube/storage-provisioner:v5
    ▪ Verwende Image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Einige Dashboard Features erfordern das metrics-server addon. Um alle Features zu aktivieren:

        minikube addons enable metrics-server

🌟  Addons aktiviert: storage-provisioner, dashboard, default-storageclass
🏄  Fertig! kubectl ist jetzt für die standardmäßige (default) Verwendung des Clusters "minikube" und des Namespaces "default" konfiguriert
(base) jan.stehr@mac-K575TX3J5F ~ % kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   18d   v1.31.0
(base) jan.stehr@mac-K575TX3J5F ~ % kubectl describe node minikube
Name:               minikube
Roles:              control-plane
Labels:             beta.kubernetes.io/arch=arm64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=arm64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
                    minikube.k8s.io/commit=210b148df93a80eb872ecbeb7e35281b3c582c61
                    minikube.k8s.io/name=minikube
                    minikube.k8s.io/primary=true
                    minikube.k8s.io/updated_at=2024_12_14T08_28_39_0700
                    minikube.k8s.io/version=v1.34.0
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
Annotations:        kubeadm.alpha.kubernetes.io/cri-socket: unix:///var/run/cri-dockerd.sock
                    node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Sat, 14 Dec 2024 08:28:36 +0100
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  minikube
  AcquireTime:     <unset>
  RenewTime:       Wed, 01 Jan 2025 13:15:29 +0100
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----             ------  -----------------                 ------------------                ------                       -------
  MemoryPressure   False   Wed, 01 Jan 2025 13:14:58 +0100   Sat, 14 Dec 2024 08:28:35 +0100   KubeletHasSufficientMemory   kubelet has sufficient
memory available
```

K8s zum Ausprobieren

kubernetes | default | Suchen

Cluster > Nodes > minikube

Konfiguration und Datenspeicheru

- Config Maps N
- Persistent Volume Claims N
- Secrets N
- Storage Classes

Cluster

- Cluster Role Bindings
- Cluster Roles
- Events N
- Namespaces
- Network Policies N
- Nodes
- Persistent Volumes
- Role Bindings N
- Roles N
- Service Accounts N

Custom Resource Definitions

Einstellungen

Systeminformation

Maschinen-ID	UUID des Systems
a4d6538c6da043c099b31ae3763635cc	a4d6538c6da043c099b31ae3763635cc

Boot ID	Kernel-Version	Betriebssystem	Version der Container-Laufzeitumgebung
359ad11b-791d-4699-9886-dc0d215a6f2a	6.10.14-linuxkit	Ubuntu 22.04.4 LTS	docker://27.2.0

Kubelet-Version	Betriebssystem	Architektur	CPU capacity	Memory capacity	Pods capacity
v1.31.0	linux	arm64	10,00	7,65Gi	110

Zuordnung

CPU		Speicher		Pods
7.5% Requests	0.0% Limits	2.2% Requests	2.2% Limits	8.2% Allocation
Cores: 0,75	Cores: 0	MiB: 170	MiB: 170	Pods: 9

K8s als „Betriebssystem der Cloud“

Das Betriebssystem abstrahiert für uns die lokalen Hardware-Ressourcen und die Komplexität der einzeln verbauten Komponenten.

Wir greifen nicht direkt darauf zu, sondern über den Kernel und seine Dienste. Dazu nutzen wir System Calls. Ein Rechner sieht also durch diese einheitliche Schnittstelle für unsere Anwendung immer gleich aus: Speicher, Netzwerk etc.

Der Scheduler steuert die Prozesse mit ihren Threads, wann und wie lange sie auf welchem Kern ausgeführt werden.

Der Kernel isoliert die Prozesse voneinander, so dass ein Absturz nicht das Gesamtsystem gefährdet, startet Anwendungen auch neu.

Kubernetes abstrahiert für uns die Netzwerkstrukturen und Rechnerknoten zu einem einheitlichen Cluster.

Wir verteilen unsere Anwendungen nicht direkt auf die einzelnen Knoten und verbinden sie selbst, sondern nutzen die API der Control Plane (kube-apiserver).

Der kube-scheduler verteilt unsere Anwendungen als Container auf die Nodes zur Ausführung. Bei Abstürzen startet er sie neu.

Die Container isolieren die einzelnen Anwendungen.

Die Control Plane bildet eine Art von Kernel für die verteilten Ressourcen des Clusters.